Stage 2 Execution Plan — Baseline Model Development (RGB & Thermal)

Time budget: 4 hours 25 minutes | **Weightage:** 15% | **Team:** all 4 members in parallel
Framework: MMDetection

Before anything: what "done" looks like (keep this visible)

By the end of this block, you need these deliverables:

1. Working MMDetection dataset configs for RGB-only and Thermal-only splits
2. A trained RGB baseline detector (checkpoint + logs)
3. A trained Thermal baseline detector (checkpoint + logs)
4. COCO-style evaluation for both — overall mAP **and** AP_small / AP_medium / AP_large
5. A low-light subset evaluation isolating RGB weakness (direct callback to Stage 1's illumination finding)
6. Error analysis: sample false-negative/false-positive visualizations per modality
7. A comparison table (RGB vs Thermal) + written recommendation for which to carry into Stage 3 fusion

Important scope note before you start: with a ~4.5 hour window for the *whole team* to set up, train two detectors, evaluate, and write up — full convergence training is not realistic. This stage produces **working baselines at reduced epoch count**, not final-quality models. Treat the checkpoints as "proof the pipeline works end-to-end + directionally correct metrics," not production models. Say this explicitly in your report so it doesn't read as an oversight.

Folder Structure (set this up first, 5 minutes)

```
medhadrishti-2026/
├── data/
│   └── vtuav_subset/              # already exists from Stage 1, untouched
├── stage1_analysis/               # already exists
├── stage2_baseline/
│   ├── configs/
│   │   ├── _base_/                # symlink or copy relevant mmdet base
configs
│   │   ├── rgb_baseline.py
│   │   └── thermal_baseline.py
│   ├── scripts/
│   │   ├── 01_setup_env.sh
│   │   ├── 02_prepare_configs.py
│   │   ├── 03_verify_dataset_registration.py
│   │   ├── 04_train_rgb.sh
│   │   ├── 05_train_thermal.sh
│   │   ├── 06_evaluate.py
│   │   ├── 07_lowlight_subset_eval.py
│   │   └── 08_error_analysis.py
│   ├── work_dirs/                 # mmdet writes checkpoints + logs here
(gitignore this)
│   └── outputs/
│       ├── metrics/
│       │   ├── rgb_metrics.json
│       │   ├── thermal_metrics.json
│       │   └── lowlight_subset_metrics.json
│       └── visualizations/
│           └── error_analysis/
└── docs/
    └── stage2_report_section.md
```

Create this now, push it, everyone pulls before starting their piece. Add `stage2_baseline/work_dirs/` to `.gitignore` — checkpoints are large and shouldn't go in git.

STEP 0 (0:00–0:20) — Environment setup, whole team in parallel

Owner: Whoever has a working CUDA + GPU setup already (fastest to unblock)

Install MMDetection and dependencies. Use `mim` (OpenMMLab's package manager) — it handles the mmcv/mmengine version matching that normally causes headaches:

```
bash
```

```
pip install -U openmim
mim install mmengine
mim install "mmcv>=2.0.0"
mim install mmdet
```

Verify install:

bash

```
python -c "import mmdet; print(mmdet.__version__)"
mim download mmdet --config rtmdet_tiny_8xb32-300e_coco --dest ./checkpoints
```

That last command both confirms the install works **and** pre-downloads the pretrained weights you'll fine-tune from — do this early since it's a large download and you don't want it blocking Step 4/5 later.

Message the group immediately once this completes — everyone else's config files (Step 2) assume MMDetection is importable.

STEP 1 (0:20–0:35) — Confirm dataset is MMDetection-ready

Owner: Shankar (owned dataset structure in Stage 1, already knows the real folder layout)

MMDetection's `CocoDataset` expects a COCO JSON + an image root. Since Stage 1 confirmed the annotations are already COCO-format, you mostly need to point MMDetection at the right paths — but verify these three things explicitly before writing configs:

python

```
import json

for split in ["train", "val", "test"]:
    with open(f"data/vtuav_subset/{split}/annotations.json") as f:
        data = json.load(f)
    cats = [c["name"] for c in data["categories"]]
    print(f"{split}: {len(data['images'])} images, categories={cats}")
```

Check specifically:

- Category list contains exactly the pedestrian/person class (note the exact string — you'll need it verbatim in configs)
- `file_name` field in `images` — does it include a subfolder prefix (e.g. `rgb/000001.jpg`) or just the filename? This determines whether you need one COCO JSON per

modality or one JSON + two different `data_root` values.

- Category `id` values — MMDetection is picky about 0-indexed vs 1-indexed category ids matching your `classes` tuple order.

If RGB and Thermal share one annotation file (same `image_id`s, different `file_name` prefixes), you'll need **two separate JSON copies** — one per modality — each pointing only at its own image set, since MMDetection dataloaders expect one JSON = one dataset. Write a tiny script to split if needed and note this in the report as a data-prep step.

STEP 2 (0:35–1:15) — Write RGB and Thermal dataset + model configs

Owner: Shashank

Base config choice: **RTMDet-tiny** (`rtmdet_tiny_8xb32-300e_coco`). Reasoning to state in the report: RTMDet is anchor-free (avoids anchor-tuning overhead under time pressure), has strong small-object performance out of the box relative to its size, and trains fast enough to fit a hackathon window — all directly relevant given Stage 1's finding that 75-87% of instances are small/medium.

`stage2_baseline/configs/rgb_baseline.py`:

```python
```

```python
_base_ = 'mmdetection/configs/rtmdet/rtmdet_tiny_8xb32-300e_coco.py'

data_root = 'data/vtuav_subset/'
classes = ('person',)   # confirm exact string from Step 1

model = dict(
    bbox_head=dict(num_classes=1)
)

train_dataloader = dict(
    batch_size=8,           # reduced from default 32 — small dataset, avoid ov
    dataset=dict(
        data_root=data_root,
        ann_file='train/rgb_annotations.json',
        data_prefix=dict(img='train/rgb/'),
        metainfo=dict(classes=classes)
    )
)
val_dataloader = dict(
    dataset=dict(
        data_root=data_root,
        ann_file='val/rgb_annotations.json',
        data_prefix=dict(img='val/rgb/'),
        metainfo=dict(classes=classes)
    )
)
test_dataloader = dict(
    dataset=dict(
        data_root=data_root,
        ann_file='test/rgb_annotations.json',
        data_prefix=dict(img='test/rgb/'),
        metainfo=dict(classes=classes)
    )
)

val_evaluator = dict(ann_file=data_root + 'val/rgb_annotations.json')
test_evaluator = dict(ann_file=data_root + 'test/rgb_annotations.json')

# Reduced schedule to fit the time budget — this is a baseline, not final train
max_epochs = 8
train_cfg = dict(max_epochs=max_epochs, val_interval=2)
optim_wrapper = dict(optimizer=dict(lr=0.001))  # scaled down for small batch s
```

```
load_from = 'checkpoints/rtmdet_tiny_8xb32-300e_coco_....pth'  # path from Step
```

`thermal_baseline.py` is identical except `ann_file` / `data_prefix` point at the thermal split. Write it as a copy with those two fields changed — don't duplicate logic, just diff the paths.

Note on epoch count: 8 epochs is a deliberate time-budget tradeoff, not a best practice. Say so in the report. If Step 4/5 finish early, bump this up and re-run rather than leaving it low by default.

STEP 3 (1:15–1:30) — Sanity-check configs before committing to a full run

Owner: Shashank (continues) or Sanjana if handing off

Run a 1-iteration dry run to catch config errors before burning training time:

bash

```
python mmdetection/tools/train.py stage2_baseline/configs/rgb_baseline.py \
    --work-dir stage2_baseline/work_dirs/rgb_baseline_test \
    --cfg-options train_cfg.max_epochs=1 train_cfg.val_interval=1
```

Confirm: no shape mismatches, no "0 images found" errors, loss is a real number (not `nan`) after the first few iterations. Fix here — don't discover a broken config 2 hours into a "real" run.

STEP 4 (1:30–2:30) — Train RGB baseline

Owner: Whoever has the GPU with more free memory / faster card

bash

```
python mmdetection/tools/train.py stage2_baseline/configs/rgb_baseline.py \
    --work-dir stage2_baseline/work_dirs/rgb_baseline
```

While this runs, monitor loss curves (`tail -f` the log or check TensorBoard if configured) — you're watching for loss decreasing steadily, not for a great final number. If loss is flat or exploding by epoch 2-3, stop and check the LR/batch-size combo rather than waiting out the full budget.

If you have two GPUs across the team: start Step 5 (Thermal) in parallel right now instead of waiting. If only one GPU is available, Thermal training happens sequentially after this finishes — plan the write-up work (Step 8 prep) during this window instead of idling.

STEP 5 (2:30–3:15) — Train Thermal baseline

Owner: Sanjana (or whoever's free, depending on parallel/sequential GPU situation above)

```bash
python mmdetection/tools/train.py stage2_baseline/configs/thermal_baseline.py \
    --work-dir stage2_baseline/work_dirs/thermal_baseline
```

Same monitoring approach as Step 4. One thing to watch specifically for thermal: since Stage 1 noted thermal images lack fine texture, don't be surprised if early-epoch loss is noisier than RGB — that's expected, not necessarily a bug.

STEP 6 (3:15–3:45) — Evaluate both models with per-scale metrics

Owner: You

```bash
python mmdetection/tools/test.py stage2_baseline/configs/rgb_baseline.py \
    stage2_baseline/work_dirs/rgb_baseline/epoch_8.pth \
    --work-dir stage2_baseline/outputs/metrics/rgb \
    --out stage2_baseline/outputs/metrics/rgb_results.pkl

python mmdetection/tools/test.py stage2_baseline/configs/thermal_baseline.py \
    stage2_baseline/work_dirs/thermal_baseline/epoch_8.pth \
    --work-dir stage2_baseline/outputs/metrics/thermal \
    --out stage2_baseline/outputs/metrics/thermal_results.pkl
```

MMDetection's default COCO evaluator already reports `AP`, `AP50`, `AP75`, `AP_small`, `AP_medium`, `AP_large` — no extra scripting needed here, just make sure you save the printed table into `outputs/metrics/rgb_metrics.json` and `thermal_metrics.json` (copy the console output or redirect `test.py`'s JSON dump).

This is the metric split that matters most given Stage 1's scale distribution — **lead with AP_small in your comparison, not overall mAP**, since overall mAP can look fine while small-object detection (the dataset's actual challenge) quietly underperforms.

STEP 7 (3:45–4:05) — Low-light subset evaluation

Owner: You (continues) or Shankar if you need to start the writeup

This is the step that turns Stage 1's "illumination is a problem" observation into a measured number. Flag dark RGB images (simple mean-brightness threshold is enough for a baseline-stage check):

```python
import cv2, json, numpy as np

with open("data/vtuav_subset/test/rgb_annotations.json") as f:
    coco = json.load(f)

dark_image_ids = []
for img_info in coco["images"]:
    img = cv2.imread(f"data/vtuav_subset/test/rgb/{img_info['file_name']}", cv2
    if img.mean() < 60:   # threshold — eyeball a few examples and adjust
        dark_image_ids.append(img_info["id"])

# Build a filtered COCO JSON containing only dark_image_ids + their annotations
filtered = {
    "images": [i for i in coco["images"] if i["id"] in dark_image_ids],
    "annotations": [a for a in coco["annotations"] if a["image_id"] in dark_ima
    "categories": coco["categories"]
}
with open("stage2_baseline/outputs/metrics/lowlight_subset.json", "w") as f:
    json.dump(filtered, f)

print(f"Flagged {len(dark_image_ids)} low-light images out of {len(coco['images
```

Re-run `test.py` on the RGB model pointing `--cfg-options test_dataloader.dataset.ann_file=...lowlight_subset.json` and compare AP against the full test set. Expect a visible drop — that drop **is** your quantified justification for thermal fusion in Stage 3. Note the exact numbers in the report.

STEP 8 (4:05–4:20) — Error analysis + report writeup

Owner: You

Pull a handful of false negatives/positives per modality for visual inclusion — MMDetection's `tools/analysis_tools/analyze_results.py` does this directly:

```bash
python mmdetection/tools/analysis_tools/analyze_results.py \
    stage2_baseline/configs/rgb_baseline.py \
    stage2_baseline/outputs/metrics/rgb_results.pkl \
    stage2_baseline/outputs/visualizations/error_analysis/rgb \
    --topk 10
```

Repeat for thermal. While the team finishes final runs, draft `docs/stage2_report_section.md`:

```markdown
## Stage 2: Baseline Model Development

### Model & Training Setup
- Architecture: RTMDet-tiny, COCO-pretrained, fine-tuned separately on RGB and
- Training: [X] epochs, batch size [X], due to time budget — see caveat below

### Baseline Results

| Modality | AP | AP50 | AP_small | AP_medium | AP_large |
|----------|----|----|----------|-----------|----------|
| RGB      |    |    |          |           |          |
| Thermal  |    |    |          |           |          |

### Low-Light Subset Results (RGB only)
[Full-set AP_small vs low-light-subset AP_small — the drop quantifies Stage 1's

### Error Analysis
[2-3 sentences + reference to saved visualizations: what RGB misses that Therma

### Recommendation for Stage 3
[Which modality/architecture choice to carry forward, backed by the numbers abo
```

Fill in brackets as results land.

STEP 9 (4:20–4:25) — Final compile and push

- Confirm both checkpoints, both metrics JSONs, and error-analysis visualizations exist in `stage2_baseline/outputs/`
- Push everything (checkpoints can go to a release/LFS or just stay local + report the numbers if repo size is a concern — don't block the push on a slow large-file upload)
- Quick team check-in: does the low-light AP drop actually look meaningful, or is it noise from a tiny subset? If the flagged low-light set has <20 images, say so explicitly rather than treating the number as solid — small-sample metrics are easy to over-read.

If something breaks (common issues, fast fixes)

`CUDA out of memory`: Drop `batch_size` in the dataloader config to 4 or 2 rather than fighting it — a smaller batch with a proportionally lower LR still produces a usable baseline.

Category id mismatch (`IndexError` or all-zero AP): Almost always caused by COCO category ids not being 0-indexed to match `num_classes=1`. Check `data["categories"][0]["id"]` — if it's `1` not `0`, MMDetection's `CocoDataset` usually handles the remapping automatically via `metainfo=dict(classes=...)`, but if you hand-built the JSON split in Step 1, double check this didn't get lost.

Training loss is `nan` after a few iterations: Almost always LR too high for the batch size after you scaled batch size down in Step 2 — halve the LR and retry the Step 3 dry run before committing to a full Step 4/5 run.

Thermal images are single-channel but config expects 3-channel input: Check if your Thermal images are already saved as 3-channel grayscale-replicated JPEGs (common in RGBT datasets) — if genuinely single-channel, you'll need to add a channel-replication step in the pipeline (`dict(type='LoadImageFromFile', color_type='color')` should force 3-channel load; verify this doesn't distort thermal values before trusting it).

Running low on time: Deliverables #2, #3, and #4 (RGB checkpoint, Thermal checkpoint, per-scale metrics) are the highest-value, most concretely gradable items — prioritize finishing training + evaluation fully even if error analysis (#6) ends up brief. A working baseline with real numbers beats a polished write-up around incomplete training.